

Sprawozdanie z Listy 4 (Laboratorium)

Obliczenia Naukowe

Agnieszka Głuszkiewicz

1 Zadanie 1: Obliczanie ilorazów różnicowych

1.1 Opis problemu

Zadanie polegało na zaimplementowaniu funkcji `ilorazyRoznicowe`, która dla zadanych węzłów interpolacji $x_0, \dots, x_n$ oraz wartości funkcji $f(x_0), \dots, f(x_n)$ obliczy ilorazy różnicowe potrzebne do skonstruowania wielomianu interpolacyjnego w postaci Newtona. Należało napisać funkcję bez użycia tablicy dwuwymiarowej (macierzy).

1.2 Wstęp teoretyczny i opis algorytmu

1.2.1 Formuła rekurencyjna

Ilorazy różnicowe definiuje się rekurencyjnie. Iloraz rzędu zerowego to po prostu wartość funkcji w węźle:

$$f[x_i] = f(x_i)$$

Iloraz różnicowy rzędu k definiowany jest wzorem:

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Wielomian interpolacyjny Newtona wykorzystuje ilorazy oparte na początkowych węzłach, tzn. $f[x_0]$, $f[x_0, x_1]$, ..., $f[x_0, \dots, x_n]$.

1.2.2 Algorytm „w miejscu”

Standardowe podejście z tablicą dwuwymiarową ma złożoność pamięciową $O(n^2)$. Aby zredukować ją do $O(n)$, zauważam, że do obliczenia ilorazów rzędu j -tego potrzebujemy jedynie ilorazów rzędu $(j-1)$ -tego. Algorytm działa na pojedynczym wektorze fx , początkowo przechowującym wartości $f(x_i)$. Aby nie nadpisać danych potrzebnych w kolejnych krokach iteracji, pętla wewnętrzna musi przebiegać **od końca** (od n do j). Sposób działania:

1. Inicjalizacja wektora fx wartościami funkcji: $fx_i \leftarrow f(x_i)$.
2. Pętla zewnętrzna po rzędach ilorazów j od 1 do n (w kodzie Julii od 2 do $n+1$).
3. Pętla wewnętrzna po indeksach i idąca wstecz od n do j .
4. Aktualizacja wartości wg wzoru:

$$fx_i \leftarrow \frac{fx_i - fx_{i-1}}{x_i - x_{i-j}}$$

Dzięki iteracji wstecznej, w momencie obliczania nowej wartości fx_i (która staje się ilorazem rzędu j), wartość fx_{i-1} wciąż przechowuje iloraz rzędu $j-1$, który jest niezbędny do obliczeń.

Poniższa tabela przedstawia zmianę zawartości wektora `fx` w kolejnych krokach algorytmu. Pogrubioną czcionką zaznaczone są elementy, które w danej iteracji osiągną swoją ostateczną wartość (współczynniki wielomianu interpolacyjnego).

Table 1: Zawartość wektora $\mathbf{f}x$ w poszczególnych etapach obliczeń

Etap	Indeks 1	Indeks 2	Indeks 3	...	Indeks n	Indeks $n + 1$
Start	$\mathbf{f}[\mathbf{x}_0]$	$f[x_1]$	$f[x_2]$	...	$f[x_{n-1}]$	$f[x_n]$
$j = 1$	$f[x_0]$	$\mathbf{f}[\mathbf{x}_0, \mathbf{x}_1]$	$f[x_1, x_2]$	...	$f[x_{n-2}, x_{n-1}]$	$f[x_{n-1}, x_n]$
$j = 2$	$f[x_0]$	$f[x_0, x_1]$	$\mathbf{f}[\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2]$	...	$f[x_{n-3}, x_{n-2}, x_{n-1}]$	$f[x_{n-2}, x_{n-1}, x_n]$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
$j = n - 1$	$f[x_0]$	$f[x_0, x_1]$	$f[x_0, x_1, x_2]$	...	$\mathbf{f}[\mathbf{x}_0, \dots, \mathbf{x}_{n-1}]$	$f[x_1, \dots, x_n]$
$j = n$	$f[x_0]$	$f[x_0, x_1]$	$f[x_0, x_1, x_2]$	...	$f[x_0, \dots, x_{n-1}]$	$\mathbf{f}[\mathbf{x}_0, \dots, \mathbf{x}_n]$

Widać tutaj, że po wykonaniu j -tej pętli zewnętrznej, $j + 1$ pierwszych elementów wektora zawiera już docelowe ilorazy różnicowe $f[x_0, \dots, x_k]$, podczas gdy pozostała część wektora przechowuje tymczasowe ilorazy rzędu j oparte na późniejszych węzłach, które posłużą do obliczeń w następnym kroku. Po zakończeniu działania algorytmu, wektor $\mathbf{f}x$ zawiera na przekątnej szukane współczynniki wielomianu Newtona: $f[x_0, \dots, x_k]$.

1.3 Rozwiązanie

Zaimplementowałam funkcję `ilorazyRoznicowe` w module `InterpolationMethods`. Funkcja przyjmuje wektor węzłów $\mathbf{x}$ i wektor wartości interpolowanej funkcji $\mathbf{f}$, a zwraca wektor ilorazów różnicowych $\mathbf{f}x$.

- **Złożoność obliczeniowa:** $O(n^2)$ (dwie zagnieżdżone pętle).
- **Złożoność pamięciowa:** $O(n)$ (operacje na kopii wektora wejściowego, bez dodatkowych tabel/macierzy).

2 Zadanie 2: Obliczanie wartości wielomianu Newtona

2.1 Opis problemu

Celem zadania było zaimplementowanie funkcji `warNewton`, która oblicza wartość wielomianu interpolacyjnego stopnia n w postaci Newtona dla zadanego punktu t . Należało zastosować uogólniony algorytm Hornera, by osiągnąć złożoność obliczeniową $O(n)$.

2.2 Wstęp teoretyczny i opis algorytmu

2.2.1 Idea algorytmu (Postać zagnieżdżona)

Wielomian interpolacyjny w postaci Newtona dany jest wzorem:

$$p_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots + c_n(x - x_0)(x - x_1) \dots (x - x_{n-1})$$

gdzie $c_i = f[x_0, \dots, x_i]$ to ilorazy różnicowe obliczone w Zadaniu 1.

Aby obliczyć wartość tego wielomianu w czasie liniowym, stosuję tę samą ideę, co w klasycznym schemacie Hornera. Wyciągam przed nawias czynniki $(x - x_k)$, tworząc zagnieżdżoną strukturę:

$$p_n(x) = c_0 + (x - x_0)(c_1 + (x - x_1)(c_2 + \dots + (x - x_{n-2})(c_{n-1} + (x - x_{n-1})c_n) \dots))$$

Dzięki takiemu zapisowi można uniknąć wielokrotnego mnożenia tych samych czynników.

2.2.2 Algorytm

Na podstawie postaci zagnieżdżonej można zdefiniować algorytm rekurencyjny. Niech w_k oznacza wartość nawiasu na poziomie k . Obliczenia rozpoczynają się od najbardziej wewnętrznego nawiasu (czyli od współczynnika przy najwyższej potęgde):

1. **Krok początkowy:** Ustawiam wartość początkową na ostatni iloraz różnicowy:

$$w_n = c_n$$

2. **Iteracja wsteczna:** Dla $k = n - 1, n - 2, \dots, 0$ obliczam kolejne wartości według wzoru:

$$w_k = c_k + (t - x_k) \cdot w_{k+1}$$

3. **Wynik:** Szukana wartość wielomianu w punkcie t to w_0 .

2.3 Rozwiązanie

Zaimplementowałam funkcję `warNewton` w module `InterpolationMethods`. Funkcja przyjmuje wektor węzłów `x`, wektor ilorazów różnicowych `fx` i punkt `t`, a zwraca wartość wielomianu w punkcie `t`.

- **Złożoność obliczeniowa:** $O(n)$ – pętla wykonuje się n razy, wykonując stałą liczbę operacji arytmetycznych w każdym kroku.

3 Zadanie 3: Wyznaczanie współczynników postaci naturalnej

3.1 Opis problemu

Zadanie polegało na napisaniu funkcji `naturalna`, która przekształca reprezentację wielomianu interpolacyjnego z postaci Newtona na postać naturalną. Mając dane węzły interpolacji: $x_0, x_1, \dots, x_n$ i współczynniki wielomianu w bazie Newtona (ilorazy różnicowe): $c_0, c_1, \dots, c_n$, należy wyznaczyć współczynniki $a_0, a_1, \dots, a_n$ takie, że:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

Algorytm ma działać w czasie $O(n^2)$.

3.2 Wstęp teoretyczny i opis algorytmu

3.2.1 Przejście z bazy Newtona do bazy potęgowej

Wielomian w postaci Newtona dany jest wzorem:

$$p_n(x) = c_n(x - x_0) \dots (x - x_{n-1}) + \dots + c_1(x - x_0) + c_0$$

Bezpośrednie wymnażanie nawiasów jest kosztowne i trudne w implementacji. Zamiast tego stosuję podejście iteracyjne, zbliżone do schematu Hornera. Algorytm rozpoczyna się od zainicjowania wektora współczynników a wartościami współczynników Newtona (c). Następnie, iterując od przedostatniego węzła wstecz, aktualizuję współczynniki, wykorzystując zależność rekurencyjną.

3.2.2 Algorytm

Niech wektor a początkowo zawiera współczynniki Newtona. Algorytm modyfikuje ten wektor w miejscu. Istota działania pętli wynika z mnożenia wielomianu przez czynnik $(x - x_k)$. Rozważmy wyraz $a_{j+1}x^j$ mnożony przez $(x - x_k)$. Otrzymujemy:

$$a_{j+1}x^j \cdot (x - x_k) = a_{j+1}x^{j+1} - (a_{j+1} \cdot x_k)x^j$$

Drugi składnik tego równania pokazuje, że współczynnik a_{j+1} po przemnożeniu przez $-x_k$ staje się składnikiem sumy dla potęgi niższej (x^j). Dlatego w kodzie aktualizujemy współczynnik $a[j]$ (odpowiadający niższej potędze) odejmując od niego przeskalowaną wartość $a[j+1]$ (odpowiadającą potędze wyższej):

$$\underbrace{a_j}_{\text{nowe}} \leftarrow \underbrace{a_j}_{\text{stare}} - \underbrace{x_k \cdot a_{j+1}}_{\text{wpływ wyższej potęgi}}$$

Schemat działania:

1. Inicjalizacja: $a \leftarrow c$ (kopia wektora ilorazów różnicowych).
2. Pętla zewnętrzna po węzłach k od $n-1$ w dół do 1.
3. Pętla wewnętrzna po współczynnikach j od k do $n-1$.
4. Aktualizacja współczynnika:

$$a_j \leftarrow a_j - x_k \cdot a_{j+1}$$

Dzięki temu w każdej iteracji zewnętrznej przekształcam część wielomianu, aż uzyskam pełną postać naturalną $a_0 + \dots + a_n x^n$.

3.3 Rozwiązanie

Zaimplementowałam funkcję `naturalna`, przyjmującą wektor węzłów `x` oraz wektor ilorazów różnicowych `fx` i zwraca wektor obliczonych współczynników postaci naturalnej `a`.

- **Złożoność obliczeniowa:** $O(n^2)$ – algorytm zawiera dwie zagnieżdżone pętle. Zewnętrzna wykonuje się n razy, a wewnętrzna średnio $n/2$ razy.
- **Złożoność pamięciowa:** $O(n)$ – funkcja tworzy jedną kopię wektora współczynników, na której wykonuje operacje.

4 Zadanie 4: Interpolacja i wizualizacja wielomianu

4.1 Opis problemu

Zadanie polegało na napisaniu funkcji `rysujNnfx`, która wykonuje interpolację zadanej funkcji $f(x)$ na przedziale $[a, b]$ za pomocą wielomianu stopnia n w postaci Newtona, z możliwością wyboru węzłów równoodległych lub węzłów Czebyszewa, i generuje wykres funkcji oraz wielomianu.

4.2 Rozwiązanie

Zaimplementowałam funkcję `rysujNnfx`, która przyjmuje funkcję f , krańce przedziału a, b , stopień wielomianu n oraz opcjonalny parametr określający typ węzłów.

Algorytm działania funkcji:

1. W zależności od parametru `wzly`, wyznaczane są punkty:
 - węzły równoodległe: $x_k = a + k \cdot \frac{b-a}{n}$, $k = 0, \dots, n$
 - węzły Czebyszewa: $t_k = \cos\left(\frac{2k+1}{2(n+1)}\pi\right)$, $k = 0, \dots, n$,
zmapowane na odcinek $[a, b]$: $x_k = \frac{1}{2}(a+b) + \frac{1}{2}(b-a)t_k$
2. Wyznaczane są wartości funkcji w węzłach, a następnie ilorazy różnicowe za pomocą funkcji z Zadania 1.
3. **Przygotowanie wykresu:**
 - Generowana jest gęsta siatka punktów (300 punktów) w przedziale $[a, b]$.
 - Obliczane są wartości dokładne $f(x)$.

- Obliczane są wartości wielomianu w każdym punkcie siatki, używając funkcji `warNewton`.
- Funkcja korzysta z pakietu `Plots.jl` i rysuje funkcję $f(x)$ oraz wielomian interpolacyjny.
- Wykres jest zapisywany do folderu `plots` i wyświetlany.

5 Zadanie 5: Testowanie interpolacji na wybranych funkcjach

5.1 Opis problemu

Celem zadania było przetestowanie zaimplementowanej funkcji `rysujNnfx` na dwóch konkretnych przypadkach, przy użyciu węzłów równoodległych. Testy przeprowadziłam dla wielomianów stopni $n \in \{5, 10, 15\}$ dla następujących funkcji:

- $f(x) = e^x$ na przedziale $[0, 1]$
- $f(x) = x^2 \sin(x)$ na przedziale $[-1, 1]$

5.2 Rozwiązanie

Napisany kod w języku Julia (plik `ex5.jl`) wykorzystuje moduł `InterpolationMethods` do wygenerowania i zapisania wykresów dla zadanych parametrów.

5.3 Wyniki i wykresy

Poniżej przedstawione są wygenerowane wykresy interpolacji dla wszystkich badanych przypadków. Linia pomarańczowa oznacza funkcję dokładną, linia fioletowa wielomian interpolacyjny, a punkty oznaczają węzły interpolacji.

5.3.1 Funkcja $f(x) = e^x$ na przedziale $[0, 1]$

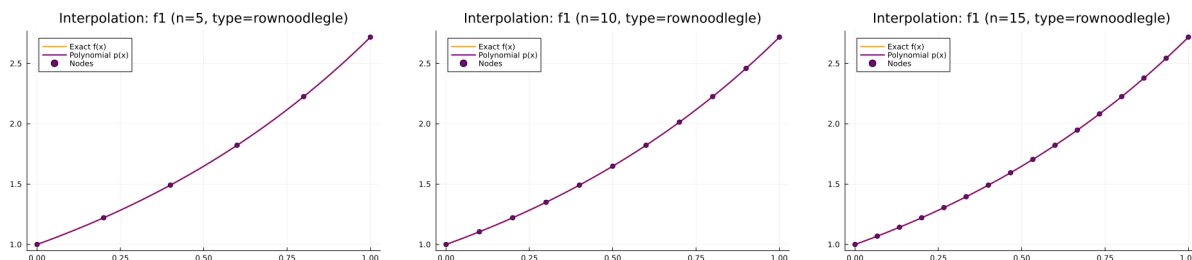


Figure 1: Interpolacja funkcji e^x na węzłach równoodległych. Od lewej: $n = 5$, $n = 10$, $n = 15$.

5.3.2 Funkcja $f(x) = x^2 \sin(x)$ na przedziale $[-1, 1]$

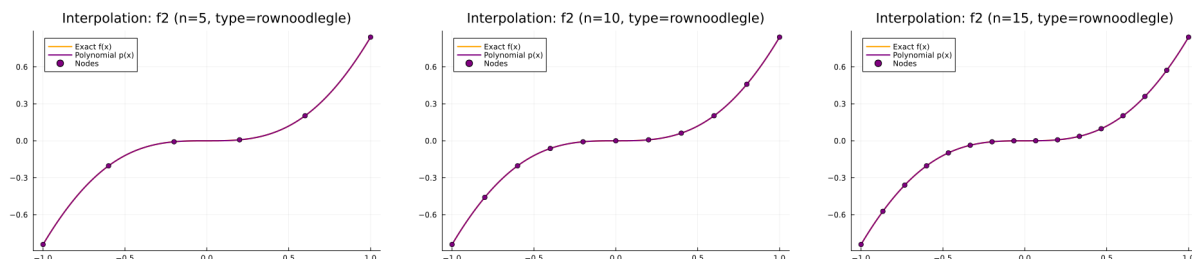


Figure 2: Interpolacja funkcji $x^2 \sin(x)$ na węzłach równoodległych. Od lewej: $n = 5$, $n = 10$, $n = 15$.

Analizując powyższe wykresy, można zauważyć, że krzywa wielomianu na tyle dobrze pokrywa się z wykresem funkcji oryginalnej, że dla rozważanych wartości n i przedziałów różnice są wizualnie niedostrzegalne.

5.4 Wnioski

Dla obu badanych funkcji (e^x oraz $x^2 \sin(x)$) na zadanych przedziałach, interpolacja wielomianowa z węzłami równoodległymi daje bardzo dobre rezultaty.

6 Zadanie 6: Analiza zjawiska rozbieżności i efektu Rungego

6.1 Opis problemu

Celem zadania było przetestowanie funkcji `rysujNnfx` dla $n \in \{5, 10, 15\}$, przy użyciu węzłów równoodległych i Czebyszewa, na dwóch przykładach:

- (a) $f(x) = |x|$ na przedziale $[-1, 1]$
- (b) $f(x) = \frac{1}{1+x^2}$ na przedziale $[-5, 5]$

6.2 Rozwiązanie

Napisany kod w języku Julia (plik `ex6.jl`) generuje wykresy dla badanych wartości n dla obu typów węzłów.

6.3 Wyniki: Funkcja $f(x) = |x|$

6.3.1 Węzły równoodległe

Przy zastosowaniu węzłów równoodległych, mimo zwiększania stopnia wielomianu, obserwujemy m.in. problemy z odwzorowaniem funkcji w punkcie $x = 0$ (miejscu załamania funkcji) oraz wyraźne oscylacje przy brzegach przedziału.

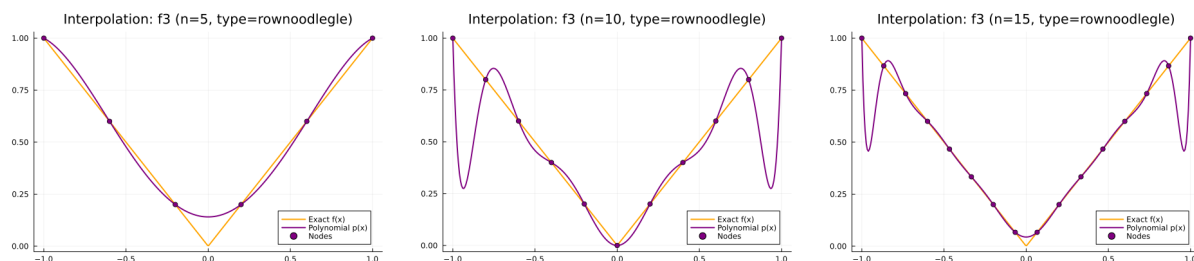


Figure 3: Interpolacja $|x|$ na węzłach równoodległych. Od lewej: $n = 5$, $n = 10$, $n = 15$.

6.3.2 Węzły Czebyszewa

Przy zastosowaniu węzłów Czebyszewa wielomian całkiem dobrze „przylega” do funkcji, choć w samym punkcie $x = 0$ widać, że wielomian ma trudność z idealnym odwzorowaniem ostrego załamania.

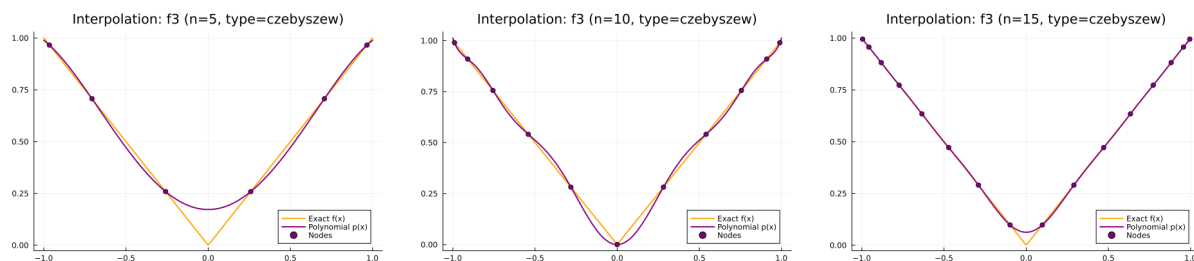


Figure 4: Interpolacja $|x|$ na węzłach Czebyszewa. Od lewej: $n = 5$, $n = 10$, $n = 15$.

6.4 Wyniki: Funkcja $f(x) = \frac{1}{1+x^2}$

6.4.1 Węzły równoodległe

Na poniższych wykresach widać nieodpowiednie zachowanie wielomianu. Wraz ze wzrostem stopnia n , wielomian coraz lepiej odwzorowuje rozważaną funkcję w środkowej części wykresu, ale na krańcach przedziału zaczyna gwałtownie oscylować, osiągając wartości znacznie wykraczające poza zakres funkcji, co opisuje się jako zjawisko Rungego.

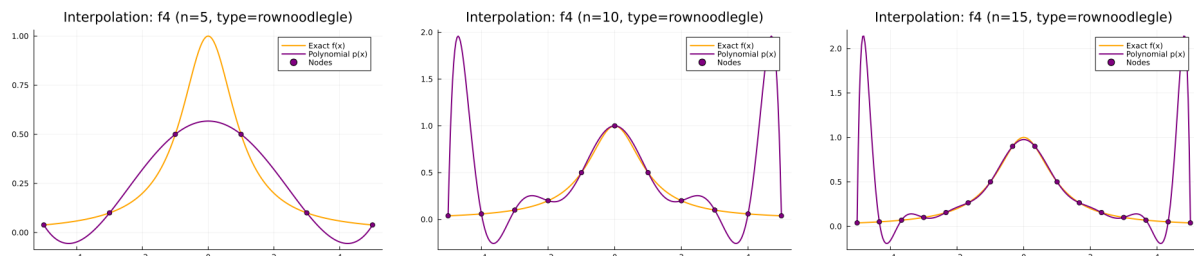


Figure 5: Interpolacja funkcji $\frac{1}{1+x^2}$ na węzłach równoodległych. Od lewej: $n = 5$, $n = 10$, $n = 15$.

6.4.2 Węzły Czebyszewa

Zastosowanie węzłów Czebyszewa sprawia, że wraz ze wzrostem stopnia n wykres wielomianu coraz bardziej przypomina wykres rozważanej funkcji.

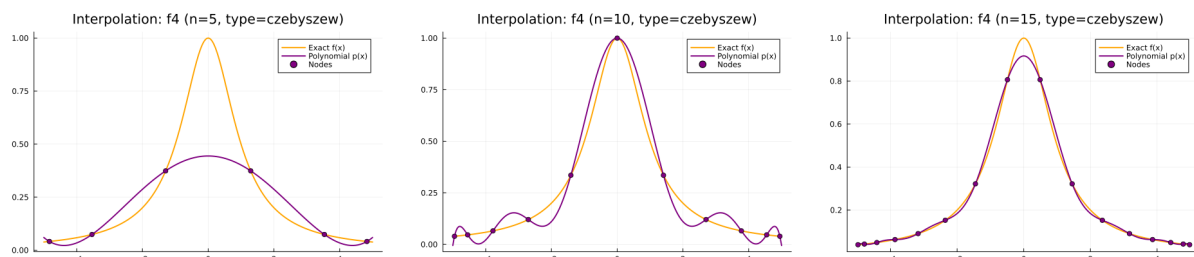


Figure 6: Interpolacja funkcji $\frac{1}{1+x^2}$ na węzłach Czebyszewa. Od lewej: $n = 5$, $n = 10$, $n = 15$.

6.5 Analiza wyników

Analiza uzyskanych wyników znajduje uzasadnienie we wspomnianym na wykładzie Twierdzeniu 2 (o błędzie interpolacji wielomianowej).

Zgodnie z tym twierdzeniem, błąd interpolacji w punkcie x zależy od iloczynu dwóch czynników:

$$E(x) \sim \underbrace{\frac{f^{(n+1)}(\zeta_x)}{(n+1)!}}_{\text{Czynnik analityczny}} \cdot \underbrace{\prod_{i=0}^n (x - x_i)}_{\text{Czynnik geometryczny}}$$

1. Czynnik analityczny: Zależy on od $(n+1)$ -szej pochodnej funkcji. W przypadku funkcji ($f(x) = \frac{1}{1+x^2}$), wyższe pochodne rosną gwałtownie wraz ze wzrostem n . Jest to cecha samej funkcji, na którą nie mamy wpływu poprzez dobór węzłów. To ten czynnik generuje potencjał do powstania dużego błędu.
2. Czynnik geometryczny: Drugi czynnik, czyli wielomian węzłowy $\omega(x) = \prod (x - x_i)$, zależy wyłącznie od wyboru punktów interpolacji. Tutaj decyduje się wielkość błędu końcowego:

- **Węzły równoodległe:** Wartość iloczynu $\prod (x - x_i)$ jest stosunkowo mała w środku przedziału, ale osiąga duże wartości na jego krańcach. Gdy pomnożymy tę dużą wartość geometryczną przez sporą wartość pochodnej, otrzymujemy bardzo duży błąd (zjawisko Rungego).

- **Węzły Czebyszewa:** Tu węzły są dobrane tak, aby zminimalizować maksymalną wartość czynnika geometrycznego na całym przedziale. Dzięki zagęszczeniu punktów na brzegach, czynnik $\prod (x - x_i)$ przyjmuje małe wartości. Ta redukcja czynnika geometrycznego jest na tyle silna, że jest w stanie zrównoważyć wzrost pochodnych funkcji, co skutkuje małym błędem interpolacji.

6.6 Wnioski

- Zastosowanie węzłów Czebyszewa jest lepszą opcją niż użycie węzłów równoodległych oraz skuteczną metodą eliminacji zjawiska Rungego.
- W przypadku funkcji, która nie jest różniczkowalna w konkretnym punkcie/punktach (jak $|x|$), zbieżność wielomianów interpolacyjnych jest powolna niezależnie od doboru węzłów, chociaż węzły Czebyszewa nadal zapewniają lepsze zachowanie na krańcach przedziału niż węzły równoodległe.